

第六章 数组

内容取自：《C Primer Plus中文版》
•10章 数组与指针



6.1 总结与回顾（数据类型与程序设计）

6.2 数组作为一种数据结构

6.3 数组的声明、操作和使用

6.4 数组作为函数参数的处理

6.5 数组的应用（排序、查找和应用）

- 折半查找
- 交换排序：冒泡排序、快速排序
- 选择排序
- 插入排序
- 循环数组：约瑟夫环



1. 高级语言的基本能力

- 简单数据类型（整型/实型/字符型）
- 程序控制结构（顺序/分支/循环、子程序调用）

2. 算法与算法设计的基本方法

- 算法是求解问题的基本思路和步骤。
- 采用良好结构化的算法设计方法。

但是，我们也遇到一些问题。

3. 求解问题的局限：

- 数据抽象的局限

- 很多客观世界的事物很难抽象到简单的数据类型上。比如：数列或级数的抽象。

- 存储能力的局限（狗熊掰棒子）

- 处理能力的局限

- 比如对任意数列的排序和查找问题，依照目前的能力尚不能够进行处理。

- 解决上述问题的基本思路：

增强数据的表达、存储和处理能力。

1. 如何表示不同类型的数据集合？

① 单一类型的元素聚集；

✓ 支持集合的定义、更新、判断从属关系、集合的交并运算。

- 集合 是 对象 无序地聚集. 例：这个班上的学生，这个教室的座椅等。
- 一个集合内包含的对象 叫做 元素，或 成员。
- 写法 $a \in A$ 的含义是 a 是集合 A 的元素。
- 如果 a 不是集合 A 的成员，写作 $a \notin A$ 。
- 描述集合：花名册法. 例： $S = \{1, 2, 3, \dots, 99\}$ 。

集合构造器，例： $S = \{x \mid x \text{ 是 } 100 \text{ 以内的正整数}\}$ 。

区间表示法，例： $[1, 99]$ 。

② 不同类型的元素聚集；

2. 如何表示不同类型的数据序列？

① 单一类型的元素序列；

✓ 支持序列的定义、更新、判断从属关系和次序关系、序列的交并运算。

序列是对象的有序列表. 例：数列、字符串。

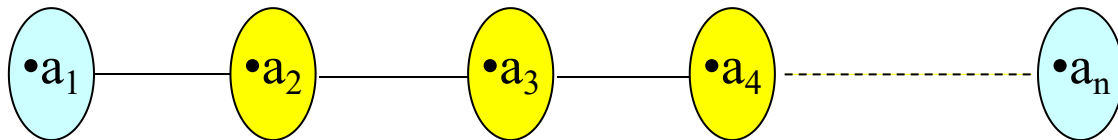
序列的数学定义是从整数子集到 S 集合的函数映射，其中的整数子集通常是 $\{0, 1, 2, 3, 4, \dots\}$ 或 $\{1, 2, 3, 4, \dots\}$ 。

可以把 a_n 看作 $f(n)$ 的等价符号， f 是从 $\{0, 1, 2, \dots\}$ 到 S 的函数。我们称 a_n 为序列项。

序列描述了 S 集合中元素之间的完全次序关系，任何两个元素都有次序关系，这种全序关系用图形表示就是线性。

② 不同类型的元素序列；

- 集合 S 上数据关系 R ，保证任意两个元素之间，要么 $a \leq b$ (a 是 b 的前驱，或 b 是 a 的后继) 要么 $b \leq a$ ，但不会同时。
- 数学上，任意两元素都可确定前后次序，可以推出任一元素在整个集合中一定有唯一位置。所有元素都按次序排列，形成一条链，线性结构。



- 数据结构的三要素：
 - 数据模型：数据类型、数据个数、数据之间的关系。
 - 整数、小数、字符编码；
 - 线性表、树、图。
 - 数据的存储方式
 - 连续存储，随机存储；
 - 数据的操作算法
 - 查找、增加、修改、删除...

6.1 总结与回顾（数据类型与程序设计）

6.2 数组作为一种数据结构

6.3 数组的声明、操作和使用

6.4 数组作为函数参数的处理

6.5 数组的应用（排序、查找和应用）



6.2 数组是连续存储的线性表

1. 线性表

- 现实世界经常存在这样的数据：某课程一个班级中按学号排列的各个学生考试成绩、一年中各月份的平均温度……其特点是这有限个数据元素的类型相同，且具有先后顺序关系；
- 上述特征的数据可以抽象为**线性表**。
- **线性表**是具有相同数据类型的 $n(n \geq 0)$ 个数据元素的有限序列，通常记为： $(a_1, a_2, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$ 其中 n 为表长， $n=0$ 时称为空表。
线性表数据元素之间为线性关系，即任意两个元素之间都可比较大小。
- **线性表有两种存储结构：连续（顺序表）、随机（链表）。**

- 线性表处理可抽象出一些基本操作，如：
 - 创建与更新：增加/删除表中的元素
 - 检索：查找表中的某个元素
 - 排序
- 一般地，线性表的程序设计采用结构化子程序设计（C语言的子函数），通过定义和调用子程序，完成线性表的各种应用。

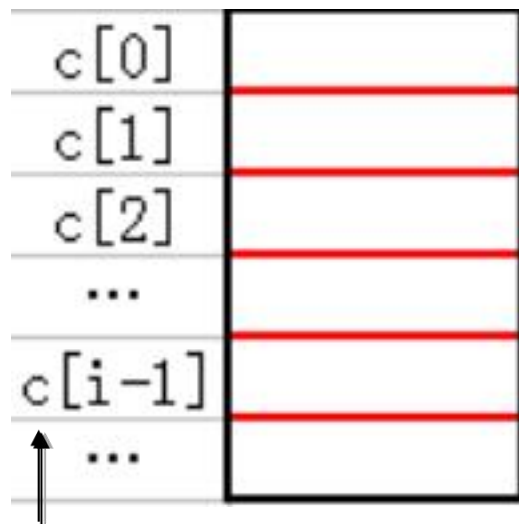
- 数组必须一次性整体定义，预先确定数据类型和元素最大个数，因为要占用一组**连续**的存储单元。

定义 元素类型名 数组名[**整型常量**表达式];

- 连续存储**：数组元素的存储单元**位置相邻**
(一个接一个)，元素具有

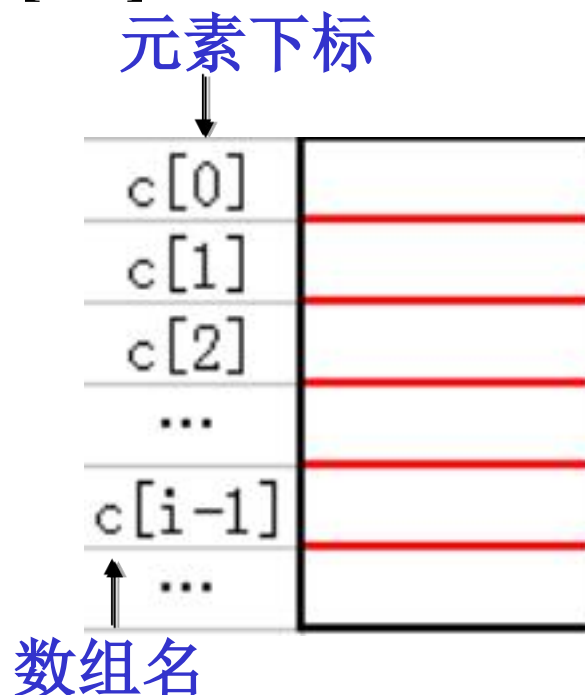
相同数据类型。

存储位置的前后表示数据
元素间的次序关系。



变量名：数组名c

- 数组必须逐个元素访问（操作）
 - 语法： **数组名[元素序号]**。其中元素序号又称下标。
 - 注意：第一个元素的序号为0，因此 $c[0]$ 引用数组 c 的第一个元素。 $c[i-1]$ 引用第 i 个元素。



数组元素操作：读取和赋值

数组元素操作，按下标读取或赋值。

- 在赋值语句**右边**时，该操作从数组元素中读取数据；

例： $x=c[1]$ ； //读取下标为1的数组元素的值，赋给变量x；

- 在赋值语句**左边**时，对数组元素进行赋值。

例： $c[2]=x*3+5$ ；//将赋值表达式右边的值赋给下标为2的数组元素。

c[0]	
c[1]	
c[2]	
...	
c[i-1]	
...	

6.1 总结与回顾（数据类型与程序设计）

6.2 数组作为顺序表的数据结构

6.3 数组的声明、操作和使用

6.3.1 数组的声明

6.3.2 数组的初始化

6.3.3 数组逐元素操作

6.4 数组作为函数参数的处理

6.5 数组的应用（排序、查找和应用）

C语言中如何使用数组呢？



6.3.1 数组的声明

一、数组的声明:

语法格式: **元素类型名 数组名[整型常量表达式];**

元素类型名 可以是整型、字符型、浮点型、结构和指针。

常量表达式说明数组元素的个数, 运算结果必须为整型。

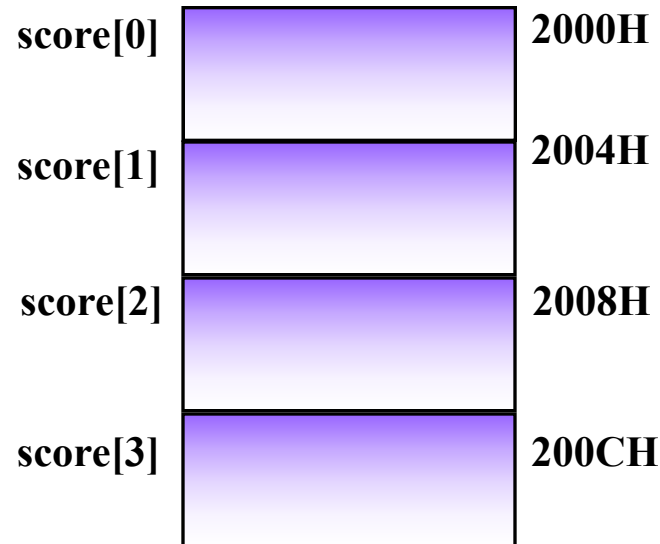
存储空间

例如:

```
int main()
{
    int score[4];
    .....
}
```

或者

```
#define SIZE 4
int main()
{
    int score[SIZE];
    .....
}
```

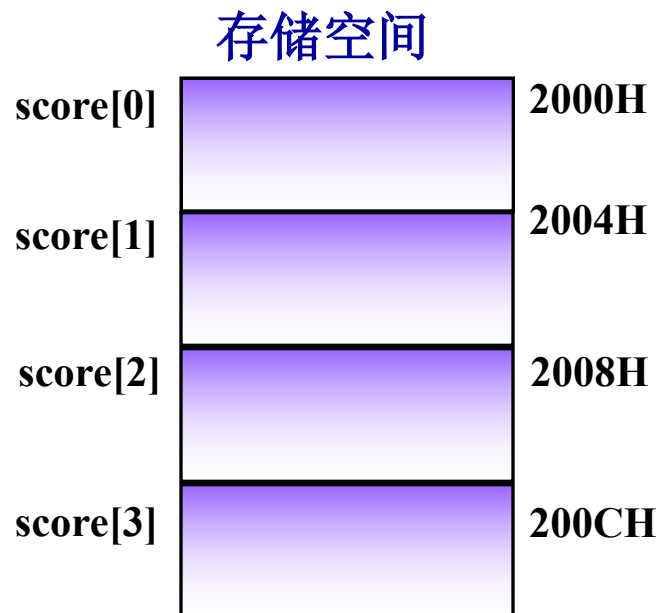


6.3.1 数组的声明

1. 数组中各元素的类型相同;
2. C语言中数组下标**从0开始**, `score[0]`表示第1个数组元素, `score[i]`表示第*i*+1个数组元素(*i*是变量)。
3. 允许在同一个类型说明中, 说明多个数组和多个变量。例如:

`int num, students[150], score[4];`

但注意数组名不能和其他变量名相同。



•数组元素的引用:

数组名[下标], 如`score[0]`;

•下标必须是整数或者整数表达式。注意: 表达式中的操作数可以是常量、变量、函数调用或表达式。

6.3.2 数组的初始化

二. 数组的初始化

C语言中可利用声明语句对数组元素的值进行初始化:

元素类型名 数组名[整型表达式]={值, 值……值};
存储空间

例如:

```
int score[4]={65, 78, 54, 91};
```

数组初始化是在编译阶段进行的。这样将减少运行时间，提高效率。

score[0]	65	2000H 2001H
score[1]	78	2002H 2003H
score[2]	54	2004H 2005H
score[3]	91	2006H 2007H

6.3.2 数组的初始化

1. 若在定义一个数组的同时赋初值，如果初始化值的个数小于数组元素的个数，剩余的元素被自动初始化为0。

例如： `int score[4]={80};`

存储空间

注意：整型数组各元素不会自动初始化为0，至少要把第一个数组元素初始化为0，才能使剩下的元素自动初始化为0。

score[0]	80	2000H 2001H
score[1]	0	2002H 2003H
score[2]	0	2004H 2005H
score[3]	0	2006H 2007H

6.3.2 数组的初始化

2. 如果初始化元素个数大于数组长度，则编译会报错，例如：

```
int score[4]={65, 78, 54, 91,60};
```

3. 如果在声明带有初始化值列表的数组时省略数组的大小，那么数组元素的个数就是初始化值列表中的元素个数。

```
int score[]={65, 78, 54, 91,60};//score有5个元素
```

4. C99新特性：在初始化列表中使用带有方括号的元素下标可以初始化某个特定的元素.

```
int score[]={[4]=60};//把score[4]初始化为60
```

```
int days[MONTHS]={31,28,[4]=31,30,31,[1]=29};,0m
```

//days元素依次为： 31,29,0,0,31,30,31,0,0,0,0,0

- 数组元素和普通的基本类型变量一样，对基本类型变量的所有操作(读、赋值、取地址等)同样适用于数组元素，基本变量能出现的地方数组元素也可以出现。

6.3.3 数组的逐元素操作

三、数组的逐元素操作：

一般情况下(字符数组例外)，无法对数组进行整体操作，例如整体赋值或者输出。要采用循环控制结构对数组的元素逐个进行操作和处理。

- C语言规定数组元素不能整体引用，每次只能引用数组的一个元素。例如，不能用赋值运算对数组进行整体赋值。因为在C语言中，数组名具有特殊含义，它代表数组的首地址。

```
int a[5];
```

```
a = {1,2,3,4,5};//错误
```

6.3.3 数组的逐元素操作

练习1. 40位学生为餐厅打分，分数分为1~10的10个等级，要求统计出各个分值的打分人数；

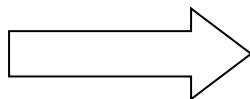
- 分析：
 - 可以循环读取40个学生的打分分值，同时进行统计，所以不需要用数组存放这40个分值；
 - 但各个分值的打分人数需要不同的累计，最后统一输出。故必须一个 $(a_1, a_2, \dots, a_{10})$ 的线性表存放。**int frequency[10]**

投票统计的算法设计

用数组保存各档统计票数

循环读取40个得分并统计

输出数组中的统计票数



定义数组a[10]

i=0; 当i<40时

输入score

a[score-1]++;

i=i+1

i=0; 当i<10时

输出i+1和a[i]

i=i+1

6.3.3 数组的逐元素操作

```
#include <stdio.h>
#define RESPONSE_SIZE 40
#define FREQUENCY_SIZE 10
int main()
{   int frequency[FREQUENCY_SIZE]={0};/*存放1~10分值票数*/
    int i, score;
    for(i=0; i<RESPONSE_SIZE; i++)
    {   scanf("%d",&score);//读取一个打分
        if (score>=1 && score<=FREQUENCY_SIZE)
            frequency[score-1]++;//统计,注意下标从0开始
    }//end for
    for(i=0; i<FREQUENCY_SIZE; i++)
        printf("%d-%d\n", i+1 ,frequency[i]);//输出结果
    return 0;
}
```

6.3.3 数组的逐元素操作

- 注意**越界控制**：不要引用超出数组范围的数组元素。出于执行速度的考虑，系统运行时不会自动检测元素下标是否越界，因此编写程序时要格外小心，由编程人员自己确保对元素的正确引用，以免因下标越界对其他存储单元中数据造成破坏。

存储空间

例：

```
int score[4]={65, 78, 54, 91};  
printf("%d", score[4]);  
/*编译不会报错，但是输出结果  
是未知的*/
```

score[0]	65	2000H
		2001H
score[1]	78	2002H
		2003H
score[2]	54	2004H
		2005H
score[3]	91	2006H
		2007H
	?	2008H
		2009H

- 6.1 总结与回顾（数据类型与程序设计）
- 6.2 数组作为顺序表的数据结构
- 6.3 数组的声明、操作和使用
- 6.4 数组作为函数参数的处理**
- 6.5 数组的应用（排序、查找和应用）



6.4 数组作为函数参数的处理

数组用作函数参数有两种形式，

- 1、将 **数组元素**(下标变量) 作为实参传递给函数；
- 2、将 **数组名** 作为实参传递给函数，目的是让被调用函数能访问、操作该数组。

数组名实际上是数组的首元素地址

```
int main()  
{  
    int array[5];  
    printf("array = %p &array[0] = %p",  
        array, &array[0]);  
    return 0;  
}
```

array = FFF0

&array[0] = FFF0

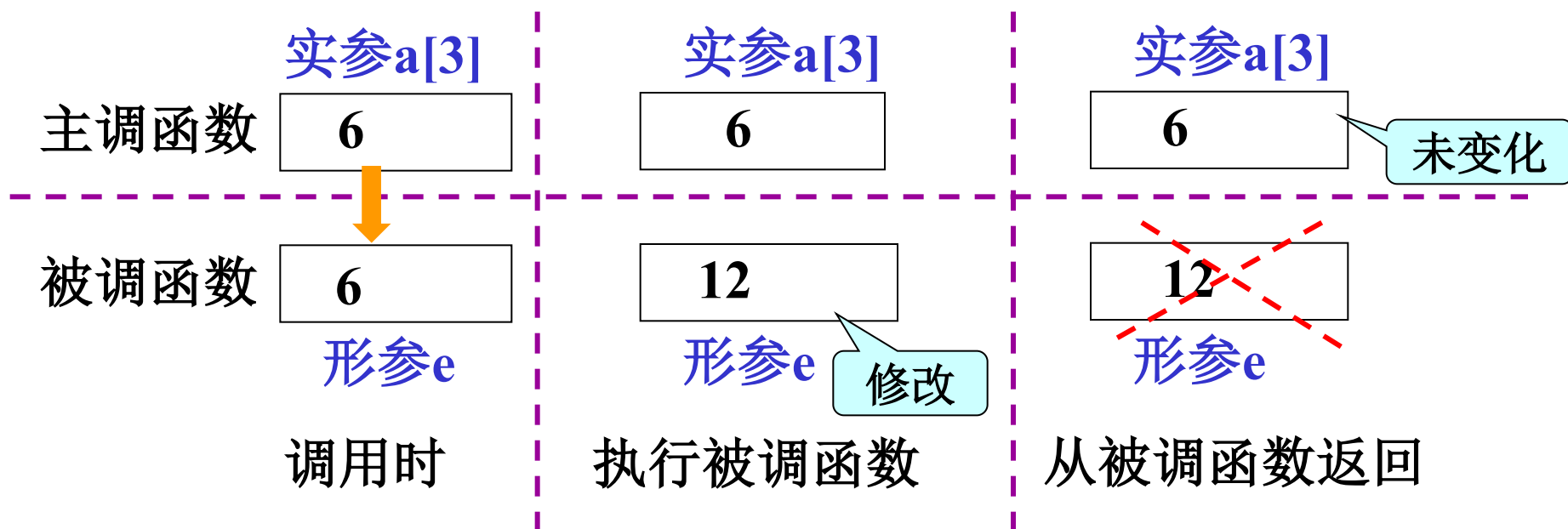
6.4 数组作为函数参数的处理

一、数组元素作函数实参

数组元素就是下标变量，它与普通变量并无区别。因此它作为函数实参使用时与普通变量完全相同，在发生函数调用时，把作为实参的数组元素的值传送给形参，实现单向的按值传送。

```
void modifyElement(int e)
{
    printf("%d", e*=2);
}
```

函数调用: `y= modifyElement (a[3]);`



数组元素作函数实参

6.4 数组作为函数参数的处理

二、将数组名作为函数的参数(形参、实参)

目的：数组名作为函数的参数时，传递的是数组的首地址（第一个元素的地址）。使得被调用函数能够访问、操作原数组！

方法：要求形参和相对应的实参都必须是类型相同的数组，都必须有明确的数组说明，此时被调函数实际操作的是原数组。同时，通常要将数组的大小传递给函数。

机理：将数组名作为实参传递给函数，函数就获得了数组的首地址，根据首地址能计算出原数组各个元素的内存地址，从而访问这些数组元素。

数组名形参-示例1

```
#include <stdio.h>
#define SIZE 5
void modify(int b[],int n);
int main()
{
    int a[SIZE]={1,2,3,4,5};
    modify(a, SIZE);
    int i;
    for (i=0;i<SIZE;i++)
        printf("%-10d",a[i]);
    return 0;
}
```

```
void modify(int b[],int n)
{
    int j;
    for (j=0; j < n; j++)
        b[j]*=2;
}
```



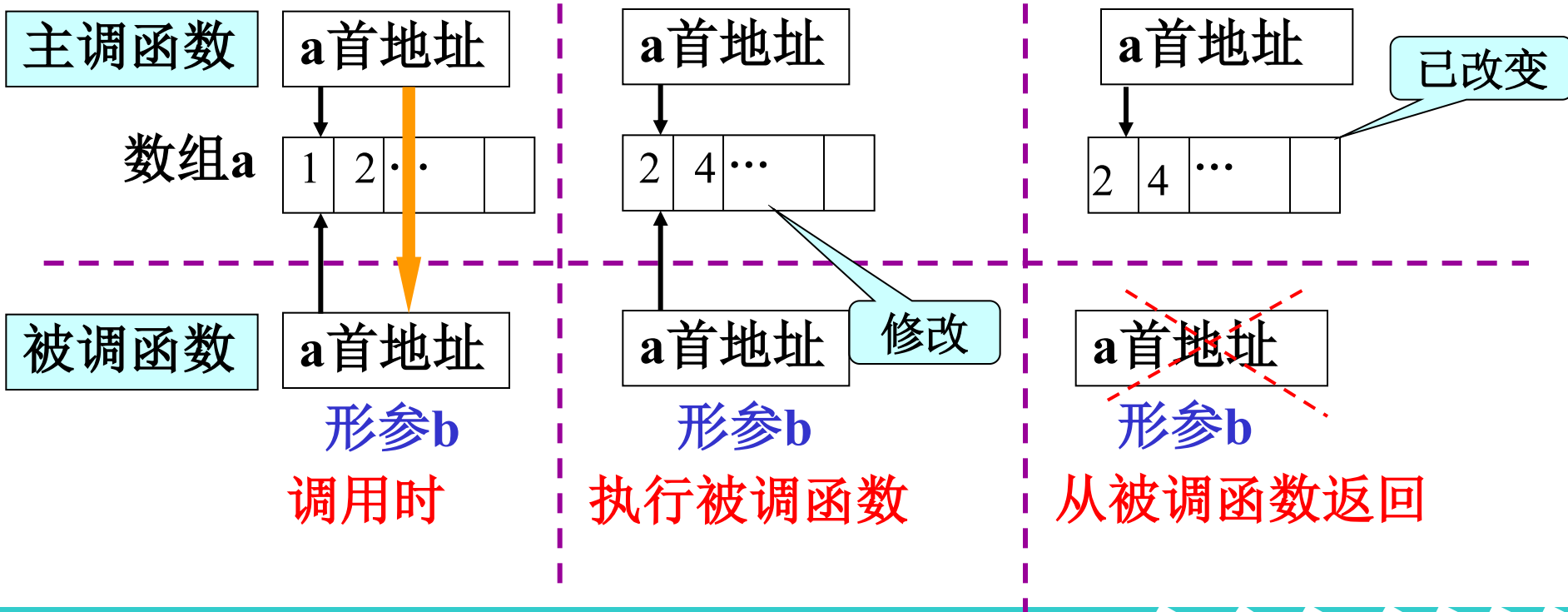
```
void modify(int b[],int n)
{
    int j;
    for (j=0;j<n;j++)
        b[j]*=2;
}
```

函数调用:

modify(a, SIZE);

如果函数要接收一个数组进行处理，则形参中必须有一个是同类型的数组，如int b[]，用于接收调用函数中数组的首地址。

- $b[j]$ 的内存地址是: $b+j*\text{sizeof}(\text{int})$
- $a[j]$ 的内存地址是: $a+j*\text{sizeof}(\text{int})$
- 而 a 和 b 的值相等，所以 $a[j]$ 和 $b[j]$ 是同一个元素





数组名形参-子函数不知道数组大小

```
#include<stdio.h>
#define SIZE 5
void sizeofArrayPara(int [],int);/*函数原型中数组名参数,
[]中不必包含数组的长度*/
int main(){
    int a[SIZE]={1,2,0,3,4};
    int i;
    printf("array a size is %zu\n",sizeof(a));
    sizeofArrayPara(a,5);
}
void sizeofArrayPara(int b[],int size){
    printf("array b size is %zu\n",sizeof(b));
    printf("%zu",size*sizeof(int));
}
```

```
array a size is 20
array b size is 8
```

数组打印的子函数设计

练习：设计一个函数`printArray`，用于输出一个整数数组中的元素

```
void printArray(const int a[], int n)
{
    int i;
    for(i = 0; i < n; i++)
        printf("%d\t", a[i]);
    printf("\n");
}
```

用`const`做数组参数的前缀，使得在 `outputArray` 函数中不允许对数组`b`的元素进行修改。

数组的基本操作3—查找元素

线性表的基本操作1—查找数组元素

设计一个函数，查找某个元素在数组中的下标并返回。

若存在多个符合条件的元素，则只返回第一个符合条件元素的下标。

函数设计考虑：

参数设计：

- 数组名为参数传入
- 数组的有效个数作为参数传入；
- 要查找的值作为参数传入；

返回结果：若找到，则返回元素下标，否则返回-1；

按值查找数组元素位置

```
int findEle(const int a[], int n, int x)
{
    int i;
    for (i=0; i<n && a[i] != x; i++)
        ;
    if (i >=n)
        return -1;        //查找失败
    else
        return i ;        //返回存储位置
}
```

标号	数据域
0	a1= 10
1	a2= 12
2	a3= 25
3	a4= 8
4	a5= 16
5	a6= 20

i = 下标初始值

i未越界且**a[i]**不符合查找要求

i++



```
#include<stdio.h>
#define SIZE 5
int findEle(int a[], int n, int x);
int main(){
    int a[SIZE]={1,2,3,4,5};
    int n,loc,x;

    scanf("%d",&x);
    loc=findEle(a,SIZE,x);
    printf("%d  loc=%d\n",x,loc);

    return 0;
}
```

6.5 数组的应用2—插入元素

- 请判断下面整型数组a的状态是否合理？
 - 将50、40、30分别存放到下标为0、1、3的数组元素中(未往下标为2的元素中存放值)。

a[0]	a[1]	a[2]	a[3]	a[4]
50	40	//	30	//

不合理！数组用于存储线性表，线性表中的元素是相邻的，这就要求它们在数组中的存储单元也要相邻。如果它们在数组中不是使用连续的存储空间，那就无法标识哪些是有效的数组元素。

6.5 数组的应用2—插入元素

➤ 线性表的基本操作2—往表中插入一个元素的函数

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
10	20	30	45		

若数组未满，且插入位置合理，则先逐个后移元素，再插入元素。

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
10	20		30	45	

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
10	20	30	45		

若数组未满，但插入位置不合理(包括数组越界)，则不允许插入元素！

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
10	20	30	45	78	60

若数组已满，则不允许插入元素！

6.5 数组的应用2—插入元素

函数设计考虑:

参数设计:

- 数组名、数组的大小作为参数传入;
- 数组中已存放的元素个数作为参数传入;
- 插入元素、插入位置作为参数传入;

返回结果: 成功插入,则返回新的元素个数;数组未
满但插入位置不合理, 返回-1;数组已满而无法插
入,返回
-2 ;

功能设计: 需要移动数组的元素, 为待插入的元素
留出位置。

数组按位置插入元素

```
int insEleByLoc(int a[], int n, int size, int i, int x){  
    int j;  
    if (n == size){  
        printf("Array is full.\n");  
        return -2;  
    }  
    if ( i<0 || i>n ){  
        printf("loc illegal.\n");  
        return -1;  
    }  
    for (j=n-1; j>=i; j--)  
        a[j+1] = a[j];  
    a[i]=x;  
    n++;  
    return n;  
}
```

标号	数据域
0	a1= 10
1	a2= 12
2	a3= 25
3	a4= 8
4	a5= 16
5	a6= 20

```
#include<stdio.h>
#define SIZE 5
void printArray(const int a[], int n);
int insEleByLoc(int a[], int n, int size, int i, int
x);
int main(){
    int a[SIZE]={1,2,3,4};
    int n,loc,x;
    scanf("%d",&x);
    n= insEleByLoc(a,4,SIZE,1,x);
    printArray(a,n);
    return 0;
}
```

6.5 数组的应用—删除元素

➤ 线性表的基本操作3—删除表中元素的函数

函数设计考虑:

参数设计:

- 数组名、数组有效元素个数作为参数传入;
- 删除元素的下标作为参数传入;

返回结果: 有效元素个数; 或参数越界提示

功能设计: 删除数组中指定下标, 需要移动数组的元素, 将待删元素后面的元素依次往前移一个位置。

a[0]	a[1]	a[2]	a[3]	a[4]
10	20	30		
a[0]	a[1]	a[2]	a[3]	a[4]
10	30			

删除元素

数组元素按位置删除

```
int delEleByLoc(int a[], int n, int i)
{ int j;
  if (n == 0)
    { printf("Array is null.\n");
      return (-1) ;}

  if (i<0 || i> n-1)
    { printf("location illegal.\n");
      return (-1); }

  for (j=i+1; j<n; j++)
    a[j-1] = a[j]; //前移

  n--; //数组长度减1
  return n;
}
```

标号	数据域
0	a1= 10
1	a2= 12
2	a3= 25
3	a4= 8
4	a5= 16
5	a6= 20

```
#include<stdio.h>
#define SIZE 5
void printArray(const int a[], int n);
int findEle(int a[], int n, int x);
int delEleByLoc(int a[], int n, int i);
int main(){
    int a[SIZE]={1,2,3,4,5};
    int n,loc,x;
    scanf("%d",&x);
    loc=findEle(a,SIZE,x);
    printf("%d  loc=%d\n",x,loc);
    n=delEleByLoc(a,SIZE,x);
    printArray(a,n);
    return 0;
}
```

练习：设计函数

```
int delEleByVal(int a[],int n,int x);
```

将数组a中所有值等于x的元素删除。



```
int delEleByVal(int a[],int n,int x)
{
    int i,j;
    for (i=0;i<n; )
        if (a[i]==x){
            for(j=i+1;j<n;j++)
                a[j-1]=a[j];
            n--;//删除元素后修正数组长度
        } else i++;

    return n;
}
```



```
#include<stdio.h>
#define SIZE 5
void printArray(const int a[], int n);
int delEleByVal(int a[],int n,int x);
int main(){
    int a[SIZE]={1,1,3,3,3};
    int n,loc,x;
    scanf("%d",&x);
    n=delEleByVal(a,SIZE,x);
    printArray(a,n);
    return 0;
}
```

更好的按值删除数组元素算法

```
int delete_x_Array(int arr[], int n, int x)
{
    int newLength = 0; // 新建数组长度
    for (int i = 0; i < n; i++)
    {
        if (arr[i] != x)
        {
            arr[newLength++] = arr[i]; // 保留非匹配值
        }
    }
    return newLength;
}
```

尾插法创建数组子函数设计v1

练习：设计一个函数createArray,接收键盘输入的任何个整数,直到-1结束.

```
int createArray(int a[], int size){
    int i,num,end=0;
    printf("Enter integers to create array,end with -1. \n");
    scanf("%d", &num);
    for(i = 0; i < size && num!=-1; i++)
    {
        a[i]=num;
        scanf("%d", &num);
    }
    return i;
}
```



```
#include<stdio.h>
#define SIZE 10
void printArray(const int a[], int n);
int createArray(int a[], int size);
int main(){
    int a[SIZE]={0};
    int n;
    n=createArray(a,SIZE);
    printArray(a,n);

    return 0;
}
```

练习：设计一个函数**createArray**,接收键盘输入的任何个整数,直到换行符结束.

```
int createArrayTail(int a[], int size){
    int i,num; char ch=0;
    printf("Enter integers by a line to create
array.\n");
    for(i=0; i<size && '\n'!=ch;i++){
        scanf("%d",&num);
        a[i]=num;
        ch=getchar();
    }
    return i;
}
```

练习3. 定义一个能容纳10个元素的整形数组a，从键盘读取多个整数（换行结束）倒序存放到数组中。

分析：如何倒序？

模仿栈，每读一个整数，插入到数组a[0]，原有元素从尾到头逐个后移。

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
10	20	30	45		

头部插入元素x

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
	10	20	30	45	

头插法创建数组的子函数设计

```
int createArrayHead(int a[], int size){
    int i,num,j;    char ch=0;
    printf("Enter integers by a line to
create reverse array.\n");
    for (i=0; i<size && '\n'!=ch; i++){
        scanf("%d",&num);
        for (j=i-1;j>=0;j--)
            a[j+1]=a[j];
        a[0]=num;
        ch=getchar();
    }
    return i;
}
```



```
#include<stdio.h>
#define SIZE 10
void printArray(const int a[], int n);
int createArrayTail(int a[], int size);
int createArrayHead(int a[], int size);
int main(){
    int a[SIZE]={0},b[SIZE]={0};
    int n;
    n=createArrayTail(a,SIZE);
    printArray(a,n);
    n=createArrayHead(b,SIZE);
    printArray(b,n);
    return 0;
}
```




数组名形参的灵活处理：实参可缩小范围

```
#define SIZE 10
void printArray(const int a[],int n);
int main()
{
    int a[SIZE]={2,1,6,9,8,36,27,25,12,7};
    printArray(a,SIZE); //输出a中的所有元素
    printArray(&a[1],SIZE-1); /*输出a中第2个元素开始
的所有元素*/

    return 0;
}
```

逆序打印数组的递归算法

- 设计一个递归函数,逆序打印整数数组.

//递归逆序打印数组元素

```
void prtRevArrayRec(int a[], int n)
{
    if (n>0){
        prtRevArrayRec(&a[1],n-1);
        printf("%d\t",a[0]);
    }
}
```

数组元素求和的递归算法

- 设计一个递归函数,计算整数数组元素之和.

```
int sumOfArrayRec(int a[],int n)
{  if (n == 0)
    return 0;
    else if (n == 1)
        return a[0];
    else
        return a[0] + sumOfArrayRec(&a[1],n-1);
        //return a[n-1]+sumOfArrayRec(a,n-1);
}
```



```
#define SIZE 5
void prtRevArrayRec(int a[], int n);
int sumOfArrayRec(int a[],int n);
int main()
{
    int a[SIZE]={1,2,3,4,5};

    prtRevArrayRec(a, SIZE);
    printf("sum=%d\n",sumOfArrayRec(a,SIZE));
    return 0;
}
```

